Kristiania

PG4200-H: Data Structures and Algorithms

Date: 24. 03. 2026 – 21. 04. 2026
Group 28. Candidate numbers: 272, 209

Kristiania

Kristiania

1.General

1.1 Introduction.

During the exam for data structures and algorithms (PG4200), we were instructed to implement several sorting algorithms into an application, extract data from a dataset, and later sort the specific data with the use of sorting algorithms listed.

This application of ours uses the Java language which we have been using throughout our semester and another course earlier. The application uses Java and only Java to solve the task presented for this exam. Another reason behind the usage of java other than the courses is that Java is one of the best suited languages for implementation of algorithms as it is a class-based and object-oriented programming language that can be used to build complex applications (GeeksForGeeks. 2025).
The algorithms used in the application shall include bubble sort, both optimized and unoptimized, insertion sort, merge sort, and quick sort. We have also included the shuffled versions of some of these sorting algorithms alongside the default sorting algorithms for this dataset as it was listed in the tasks (Shuffled version rearranges the dataset randomly before sorting it which may or may not result in a different value being presented). The application only includes the algorithms that we have been tasked with using. Algorithms that we have used are already existing algorithms that have been modified to fit our application and dataset. The references for where we have gathered the algorithms from are listed in the same class where the algorithm has been used.

The purpose of this report is to reflect on the usage of different algorithms and how they behave with the included dataset. This report will cast light on how different algorithms behave in different scenarios, complexities of different algorithms, and pros and cons/ strengths and weaknesses of the algorithms.

Kristiania

1.2 Data set.

The dataset that has been used to complete these tasks is the wine quality dataset which has been downloaded from " https://archive.ics.uci.edu/dataset/186/wine+quality " better known as the UC Irvine Machine Learning Repository. This dataset contains 2 CSV files which have been reviewed and up to the current date as in the latest version available.
The data list will be referred to in the code as "WineLists" and will be stationed in a folder which can later be accessed by the application. The CSV files include information such as fixed acidity, volatile acidity, citric acid, residual sugar, chlorides, free sulfur dioxide, total sulfur dioxide, density, pH sulphates, alcohol, and quality. Most of these data are miscellaneous since the tasks demand the use of unique alcohol values. To clarify, there are 6497 instances of alcohol values and 111 instances of unique alcohol values in the CSV files.
Note that more than alcohol values have been used in some of the features in the application. For the tasks where we must implement unique algorithms, only unique alcohol content values have been used.

1.3 Code setup.

The code has been designed with both a user-friendly menu with easy instructions and a benchmark tool for further operations and testing for the sake of the report.
The code includes a Main class that directs you to the menu with a try catch. The reasoning behind this is that we have learned in the PGR112 "object-oriented programming" course that it is a good habit to get out of main.

```java
public class Main {
    public static void main(String[] args) {
        System.out.printf("---Welcome to our wine collection---\n" +
                "---What do you wish to see or do?---\n");



        //what we did here is we made sure to get out of main
        //as soon as possible since we learned its a good habbit
        //during our Java coding class during semester two.
        menu m = new menu();
        try {
            m.menu();
        } catch (RuntimeException e) {
            System.out.println("Error: " + e.getMessage());
            throw new RuntimeException(e);
        }
    }
}
```

From: src/main.java


Kristiania

Kristiania

Then the code includes a menu with 14 options that lets you choose features from the BubbleSort class, CSVApplication class, InserionSort class, MergeSort class, and QuickSort class.

```
public class menu{  2 usages
    /*this is out menu we made sure to implement a menu to make it easier
    for the instructor to navigate through the completed tasks*/
    public void menu() {  1 usage
    int choice = 0;
    Scanner menuinput = new Scanner(System.in);
    while (choice != 15) {
        System.out.println("---//// The Wine Menu ////---");
        System.out.println("what do you want to do?\n");
        System.out.println("Do you wish to print out all the wine? press0\n" +
                "Do you wish to shuffle? you can sort it later :) press1\n" +
                "Do you wish to Bubblesort(Unoptimized)? press 2\n" +
                "Do you wish to Bubblesort(Optimized)? press 3\n" +
                "Do you wish to sort a shuffled bubblesort(Unoptimized)? press 4\n" +
                "Do you wish to sort a shuffled bubblesort(Optimized)? press 5\n" +
                "Do you wish to insertionSort? press 6\n" +
                "Do you wish to sort a shuffled insertionsort? press 7\n" +
                "Do you wish to mergeSort? press 8\n" +
                "Do you wish to sort a shuffled mergeSort? press 9\n" +
                "Do you wish to quickSort (first element as pivot) ? press 10\n" +
                "Do you wish to quickSort (last element as pivot)? press 11\n" +
                "Do you wish to quickSort (random element as pivot)? press 12\n" +
                "Do you wish to quickSort (median of three as pivot)? press 13\n" +
                "Do you wish to use the Benchmark? press 14\n" +
                "Do you wish to quit the menu? press15\n" );
```

```
choice = menuinput.nextInt();
switch (choice) {
    case 0 -> reader();
    case 1 -> Shuffle();
    case 2 -> nonOptimizedBBLS();
    case 3 -> optimizedBBLS();
    case 4 -> shufflednonOptimizedBBLS();
    case 5 -> shuffledOptimizedBBLS();
    case 6 -> insertionSort();
    case 7 -> shuffledIS();
    case 8 -> mergeSort();
    case 9 -> shuffledMS();
    case 10 -> QSFirst();
    case 11 -> QSLast();
    case 12 -> QSRandom();
    case 13 -> QSMedian();
    case 14 -> WineSortingBenchmark.run();
    case 15 -> exit();
}
```

From: src/menu.java

Kristiania

This class also includes public voids that call upon the methods used by the cases listed in the figure above.
Example:

```
66      //calls upon the nonoptimized Bubble sort method in the Bubblesort class
67      //this sorts the unique wine values
68      public void nonOptimizedBBLS() {  1 usage
69          Timer timer = new Timer();
70          timer.start();
71          BubbleSort.sortUniqueAlcoholUnOpt();
72          timer.end();
73          System.out.printf("Time: %.4f ms%n", timer.durationMillis());
74          System.out.println("Time (ms); " + timer.getElapsedTime() + "ms");
75      }
```

From src/menu.java
The bubbleSort class includes two wrapper methods that create CSVApplication data loader, then fetch data by using the filereaderUnique() and later calls upon the bubbleSortUnOpt() / bubbleSortOpt() methods.

```
10          public static void sortUniqueAlcoholOpt() {  1 usage
11              CVSApplication csvApplication = new CVSApplication();
12              ArrayList<Wines> wineList = csvApplication.filereaderUnique();
13              bubbleSortOpt(wineList);
14          }
```

```
5           public static void sortUniqueAlcoholUnOpt() {  1 usage
6               CVSApplication csvApplication = new CVSApplication();
7               ArrayList<Wines> wineList = csvApplication.filereaderUnique();
8               bubbleSortUnOpt(wineList);
9           }
```

From src/BubbleSort.java

The class also includes both the optimized and the unoptimized Bubblesort that have been gathered from GeeksForGeeks. Optimized Bubblesort is from https://www.geeksforgeeks.org/dsa/bubble-sort-algorithm/ . Author GeeksForGeeks and last updated on the 8 of December 2025. The unoptimized Bubblesort is from https://www.geeksforgeeks.org/dsa/java-program-for-bubble-sort/ . Author GeeksForGeeks and last updated on the 23 July 2025.

Kristiania

The CSVApplication is a class that consists of one public void and two public array lists.

```
public void filereader() {  1 usage
public ArrayList<Wines> filereaderUnique(){  7 usages
public ArrayList<Wines> shuffledList(){  5 usages
```

From: src/CSVApplication.java

The job of this class is to read data from the two CSV files and store the data so that the algorithms and other functions can take that data and use and sort them.

The void in this class reads the two CSV files, stores them the data requested in separate lists, and later prints the data requested. Later, it prints the data, so it shows alcohol percentage and the quality of the wine. The reason this method stands out is because this is our first attempt at reading and storing data before we start on the tasks. We decided to leave it in as one of the features since it shows how we managed to optimize the code later on.

```
7       //Reads the dataset for non unique wine values
8       public void filereader() {  1 usage
9           //these are the paths for the filreader so it can access the cvs files
10          String fileWineR = "C:\\Users\\nikol\\IdeaProjects\\EksamenPrepAlg\\src\\Wines\\winequality-red.csv";
11          String fileWineW = "C:\\Users\\nikol\\IdeaProjects\\EksamenPrepAlg\\src\\Wines\\winequality-white.csv";
12
13          //these make sure the app reads files into a list
14          ArrayList<Wines> whiteWine = Wines.readWineFiles(fileWineW);
15          ArrayList<Wines> redWines = Wines.readWineFiles(fileWineR);
16
17          //prints out the value for white and red wine
18          System.out.println("Contents of white wine" + whiteWine.size());
19          for (Wines w : whiteWine) {
20              System.out.println("Alcohol perc" + w.alcohol + "%" + " // Quality of the wine " + w.quality + "/10");
21          }
22          System.out.println("\n\n\n\n\n\nContents of red wine" + redWines.size());
23          for (Wines w : redWines) {
24              System.out.println("Alcohol perc" + w.alcohol + "%" + " // Quality of the wine " + w.quality + "/10");
25          }
26      }
```

From: src/CSVApplication.java

Later, we implemented the array list that we would utilize for the tasks to come. This method essentially does the same thing with reading both CSV files and fetching data. Where the similarities end is that this method fetches the data and filters the duplicates, later combines them into one list and later returns the combined list so it can be used later.

The last method used in this class is the shuffledList() which essentially gets the unique wine list from the method before and shuffles it so that the list is random. Later, we added that it prints the size of the list. At the end, it returns the shuffled list so that it can be accessed later.

```
53      public ArrayList<Wines> shuffledList(){  5 usages
54          //fetches the wineList
55          CVSApplication cvsApplication = new CVSApplication();
56          ArrayList<Wines> wineList = cvsApplication.filereaderUnique();
57          //shuffles it so it is random
58          Collections.shuffle(wineList);
59          //prints the size
60          System.out.println("Shuffled the wines:" + wineList.size());
61          //returns the list
62          return wineList;
63      }
64  }
```

From: src/CSVApplication.java

The Insertionsort class includes the wrapper just like in the Bubblesort class which is later followed up by the insertion sort method and the credits for where we got the algorithm from. The insertion sort algorithm has been gathered from https://codegym.cc/groups/posts/insertion-sort-in-java . Author is Oleksandr Miadelets and was last updated on 14 January 2025.
The Mergesort class essentially works in on the same principle with a wrapper, followed by the merge sort method and credits. The algorithm has been gathered from https://www.geeksforgeeks.org/dsa/merge-sort/ . Author is GeeksForGeeks and was last updated on the 3 of October 2025.
The last algorithm is in the Quicksort class. This class is structured differently. It starts with a counter for the task that was listed. Then there is the Quicksort method which is a part of the Quicksort mechanism. Then there is the partition static int which has a switch inside that is crucial for the pivoting mode choosing then it calls upon swap method. Then there is the sort void which starts off the Quicksort process. Later there are 4 static voids which let you choose which pivoting mode you want.

Kristiania

1.4 Benchmark information

For the best algorithmic outcome for the questions, we were tasked to complete. We have implemented a segment in our application dedicated to testing and benchmarking the different algorithms. There are some segments in this exam that need to be proven with our own outputs that is why we made the benchmark application for our research. These tests will further solidify our arguments and choices for why we believe this is the right answer for the tasks presented to us. Due to the fact that all algorithms are slow, the first few tries you run them we have taken accountability to properly warm up each algorithm before presenting the answers for our tasks. Note that we have implemented a timer. The use of this timer will include both the time of the algorithm being used in the menu part of the application and in the benchmark part of the application. What this means is that the time when the algorithm is used in the menu application will be greater than the time when the algorithm is using standalone in the benchmark application.

1.5 Dataset structuring configuration

A valuable part of this course is to argue for how the data is structured and why we chose to structure the dataset as we did and how we did the major operations (Gupta, R (2026). LO 1_SortingAlgorithms, slide 9 and 11). Before coding and report writing, we have assessed the problem with this dataset and argued how to structure this dataset. We have concluded that focus on efficiency and scalability is to be focused during our structuring.

When it comes to choosing how to structure data, there are a few options to choose from when implementing. There is the Array, the Array list, the Linked list, stack, and queue (Gupta, R (2026). LO 1_SortingAlgorithms, slide 3). We chose to exclude the stack and queue since it did not fit the dataset. The reason we excluded the stack is the fact that it operates with the last in, first out methodology which is not what we need in this dataset since it fails to get access to the value that is behind the initial top of the stack (GeeksForGeeks. 2026). Where the queue fails is the fact that it works on the first in, first out methodology and only the first element can be accessed just. Just like in the stack data structure, it fails to extract the elements behind the first element (GeeksForGeeks. 2026). Later, we excluded the linked list due to the fact that it is worse at indexing than Arrays or Array list. It could work but it is a bad idea in the long run due to the fact that other algorithms have to access the list and manipulate it. It could work if the only thing with the data set we needed to do was insertion and deletion but with the algorithms we need swapping, comparing, and shuffling which are tasks that Arrays and Array list tend to perform better on (W3 schools. n.d). In the end, we could not decide between the array and the array list. Since both would be great candidates, we first started listing benefits of both data structures. Both array and array list have constant time access, efficient iterations, and both are very well suited for sorting algorithms that we were tasked with to implement (GeeksForGeeks. 2026, 2025, and Bharat. 2024).

In the end, we have concluded that the array list would be the better choice even if it was kind of a controversial choice. Even though the csv files are fixed, and the dataset is fixed, we would argue that the array list is still better. This is since the array list is more scalable than the array in case in the future the developer that uses our code wanted to implement more data to the dataset, it

would be far easier to do that with the array list than with the array. Another argument is the fact that when a developer uses an array to read from a csv file, you must predetermine and preallocate the size of the array to fit the csv file (Shravya. 2024).

Now the performance of both data structures is the same as they have the exact same constant time of O (1), the only difference is that the array list may be slower due to resizing (Mcgregor. 2025). On of the factors we aimed for was scalability and efficiency and after some critical thinking and argumentation we have concluded the fact that array list is slightly better for this scenario.

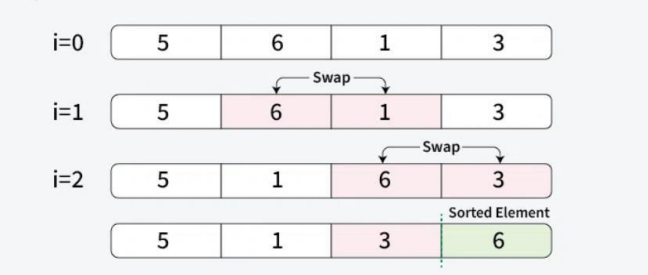
Kristiania

2. Bubble Sort

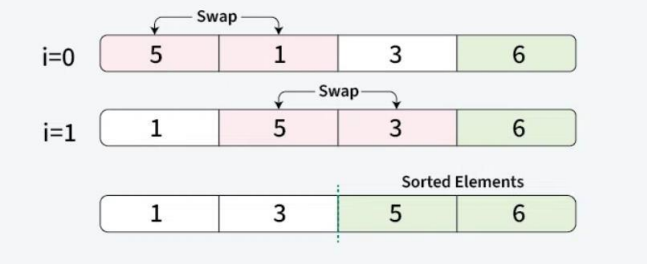
2.1 Explanation of the algorithm

Bubblesort is one of the most basic algorithms. It sorts the values by repeatedly comparing neighboring elements through a list. Bubblesort first takes one element and compares it with the next element. If the element is larger than the other element, they get swapped. The algorithm then continues to the next pair and repeats the process. After a full pass through the list, the largest element will be passed to the end, and the cycle continues until there are no more swaps needed. The outcome at the end is that the largest is to the right, and the lowest is to the left, and you get an ascending list (Salawsky. 2025).
This can be easily visualized with this illustration:
credits to geeksforgeeks. 8. Dec, 2025. Link: https://www.geeksforgeeks.org/dsa/bubble-sort-algorithm/

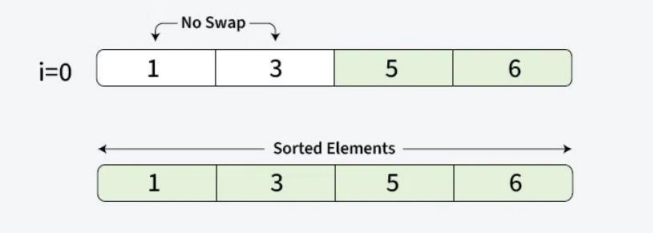
Step 1.



Step 2.



Step 3.



Kristiania

Bubblesort is the simplest sorting algorithm, and it works with smaller datasets such as the wine list in this exam. This algorithm will not be efficient for a larger dataset as the average and worst-case time complexities are very high (GeeksForGeeks. 2025). Another place not to use Bubblesort is in real-world systems where speed is a priority. Examples of these could include such as database indexing, graphics rendering, and even sorting of a large contact list on a phone (Alma Better. 2026).

For reference, a good place to implement Bubblesort would be educational purposes such as this exam or small-scale datasets. The reasoning behind this is that Bubblesort is way too simple, and that is its greatest weakness (Alma Better. 2026)

Key difference during this exam is to note that we have implemented both an optimized and a nonoptimized Bubblesort to calculate the differences in the time complexities and if the worst-case time complexity is improved in the optimized version.

The difference between optimized and nonoptimized Bubblesort is that the nonoptimized version of Bubblesort always goes through the entire list the same number of times no matter what. It keeps comparing and looping until all of the switches are completed. This wastes time and in the optimized version checks if any swaps happened during a pass. If there are no swaps, it means that a sort has already happened and moves on. This decreases the time wasted on unnecessary looping through the list.

2.2 Code

For our code, we implemented the geeksforgeeks' simple bubblesort and modified it to fit our array list. The nonoptimized bubblesort follows a method where if the alcohol value of the current element is greater than the next one, then the two elements are swapped using a temp variable. After each pass the largest values eventually "bubbles up" top the correct position and later after the bubblesort has completed its process the size of the list is being returned leading up to the values being printed

Kristiania

```java
        //this is the unoptimised bubble sort we used
        //this bubblesort has been inspired and by geeksforgeeks
        ////////////////////////////////////////////////////////
        /// author: GeeksForGeeks                            ///
        /// gathered: 26.03.2026                             ///
        /// url: https://www.geeksforgeeks.org/dsa           ///
        /// /java-program-for-bubble-sort/                   ///
        /// title: Java Program for Bubble Sort              ///
        /// last updated: 23 jul, 2025                       ///
        ////////////////////////////////////////////////////////
        public static int bubbleSortUnOpt (ArrayList<Wines> uniqueWines) {  2 usages
            int size = uniqueWines.size();

            for (int i = 0; i < size - 1; i++) {
                for (int j = 0; j < size - i - 1; j++) {
                    if (uniqueWines.get(j).alcohol > uniqueWines.get(j + 1).alcohol) {
                        Wines temp = uniqueWines.get(j);
                        uniqueWines.set(j, uniqueWines.get(j + 1));
                        uniqueWines.set(j + 1, temp);
                    }
                }

            }
            System.out.println("unique alcohol values:" + uniqueWines.size());
            for (Wines w : uniqueWines) {
                System.out.println(" Alcohol: " + w.alcohol);
            }
            return size;
        }
```

From: src/BubbleSort.java

When it comes to the optimized Bubblesort we did essentially the same thing we modified geeksforgeeks' optimized Bubblesort and what method this algorithm follows is just like the nonoptimized one. The difference is that our optimized Bubblesort has a swapped Boolean implemented where the variable tracks when a swap has occurred during the completion of a loop. At the start of the outer loop iteration swapped is set to false and if the swap happens it is set to true. After a pass, the Bubblesort checks the "swapped" if no swaps have been made, then it means the list is already sorted, and the loops breaks wasting no time and improving performance in the best-case scenario.

Kristiania

```java
    //this is the optimised bubble sort we used
    //this bubblesort has been inspired and by geeksforgeeks
    ////////////////////////////////////////////////////
    /// author: GeeksForGeeks                       ///
    /// gathered: 28.03.2026                        ///
    /// url: https://www.geeksforgeeks.org/dsa      ///
    /// /bubble-sort-algorithm/                     ///
    /// title: Bubble Sort                          ///
    /// last updated: 8 dec, 2025                  ///
    ////////////////////////////////////////////////////
    public static int bubbleSortOpt (ArrayList<Wines> uniqueWines) {  2 usages
        int size = uniqueWines.size();
        boolean swapped;
        for (int i = 0; i < size - 1; i++) {
            swapped = false;
            for (int j = 0; j < size - i - 1; j++) {
                if (uniqueWines.get(j).alcohol > uniqueWines.get(j + 1).alcohol) {

                    Wines temp = uniqueWines.get(j);
                    uniqueWines.set(j, uniqueWines.get(j + 1));
                    uniqueWines.set(j + 1, temp);
                    swapped = true;
                }
            }

            if (swapped == false)
                break;
        }
        System.out.println("unique alcohol values:" + uniqueWines.size());
        for (Wines w : uniqueWines) {
            System.out.println(" Alcohol: " + w.alcohol);
        }
        return size;
    }

}

```

from: src/BubbleSort.java

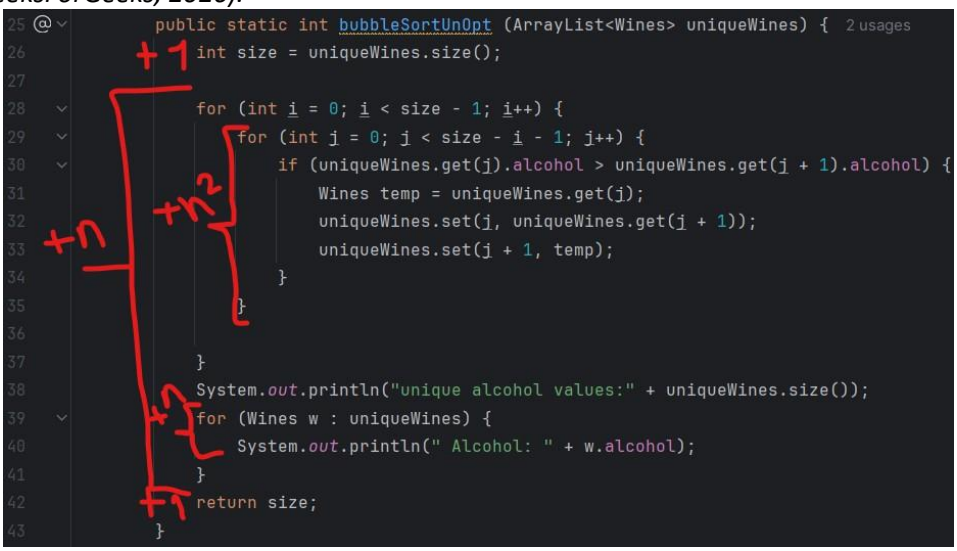
2.3 Calculation of time complexity

To calculate the time complexity of our Bubblesort one must analyze the number of operations. This means that you would have to count the number of comparisons and swaps relative to the input size (n) (w3school, n.d.).

Since both the optimized and unoptimized Bubblesort have two nested loops, we would have to calculate the n times for both the outer and the inner loops. To calculate the time-complexity of the unoptimized bubblesort we must analyze the outer loop which runs n – 1 time while the

Kristiania

inner loop runs to n - i - 1 times for each iteration. Due to the Bubblesort being unoptimized and there is no early break like the optimized one means that best, average and worst cases are all $O(n^2)$ (w3schools, n.d.). When it comes to the question of random shuffle effecting the Bubblesort here is not the case. The reason for this is because the unoptimized Bubblesort always runs full loops without any early breaks.
Info—For the space complexity due to the method only using one temporary variable, the space complexity is O(1) (GeeksForGeeks, 2026).




```
25 @        public static int bubbleSortUnOpt (ArrayList<Wines> uniqueWines) {  2 usages
26              int size = uniqueWines.size();
27
28              for (int i = 0; i < size - 1; i++) {
29                  for (int j = 0; j < size - i - 1; j++) {
30                      if (uniqueWines.get(j).alcohol > uniqueWines.get(j + 1).alcohol) {
31                          Wines temp = uniqueWines.get(j);
32                          uniqueWines.set(j, uniqueWines.get(j + 1));
33                          uniqueWines.set(j + 1, temp);
34                      }
35                  }
36
37              }
38              System.out.println("unique alcohol values:" + uniqueWines.size());
39              for (Wines w : uniqueWines) {
40                  System.out.println(" Alcohol: " + w.alcohol);
41              }
42              return size;
43          }
```

This is our calculation for time complexity for our nonoptimized bubblesort.

Best-case	Average-case	Worst-case
$O(n^2)$	$O(n^2)$	$O(n^2)$

2.4 Effect of Shuffling our Dataset

To verify the effect of shuffling our dataset with both un-optimized and optimized Bubblesort, we ran 1 000 000 repetitions with and without shuffling. The two screenshots below demonstrate the effects of shuffling our non-optimized Bubblesort. Here, the average sorting time changes when shuffling. Non shuffling led to an average sorting time of 301 µs, and with shuffling gave us an average sorting time of 337 µs

```
Algorithm      : BubbleSort (non-optimised)
Shuffle        : shuffle
Repetitions    : 1000000
total time     : 337801729µs  (337801.729ms)  (337.801729s)
Average time   : 337µs  (0.337ms)  (3.37E-4s)
```

```
Algorithm      : BubbleSort (non-optimised)
Shuffle        : not shuffle
Repetitions    : 1000000
total time     : 301076614µs  (301076.614ms)  (301.076614s)
Average time   : 301µs  (0.301ms)  (3.01E-4s)
```

To demonstrate the effect of shuffling our dataset, we ran our optimized Bubblesort implementation both with and without shuffling, as shown below. Without shuffling, we observed a lower average time of 284µs, whereas enabling shuffling increased the average time to 303µs. This shows that the effect of shuffling the list only makes the algorithm slightly slower, even though the asymptotic time complexity remains $O(n^2)$ in both with and without shuffling.

```
Algorithm      : BubbleSort (optimised)
Shuffle        : shuffle
Repetitions    : 1000000
total time     : 303214367µs  (303214.367ms)  (303.214367s)
Average time   : 303µs  (0.303ms)  (3.03E-4s)
```

```
Algorithm      : BubbleSort (optimised)
Shuffle        : not shuffle
Repetitions    : 1000000
total time     : 284554286µs  (284554.286ms)  (284.554286s)
Average time   : 284µs  (0.284ms)  (2.84E-4s)
```

Kristiania

For the optimized Bubblesort, the best-case time complexity is O(n), since it has a break if the list is sorted or not shuffled. Due to the Bubblesort having nested loops, the average and worst-case complexities are both $O(n^2)$ **since th**e algorithm still needs to perform multiple passes through the list. When it comes to the optimized Bubblesort shuffling the list effects when the algorithm breaks the sort. Shuffling the optimized Bubblesort makes the algorithm behave like an unoptimized Bubblesort with an $O(n^2)$ time complexity.
Info—the space complexity is O(1), since only a temporary variable is used during swapping.

This is our calculation for our optimized bubblesort unshuffled

Best-case	Average-case	Worst-case
$O(n)$	$O(n^2)$	$O(n^2)$

Shuffled

Best-case	Average-case	Worst-case
$O(n^2)$	$O(n^2)$	$O(n^2)$

2.5 Conclusion

To conclude, both optimized and unoptimized bubble sorts have $O(n^2)$ time complexity when it comes to average and worst-case scenarios. This is changed when it comes to the optimized bubblesort since it improves the time complexity $O(n)$ when the list is presorted or not shuffled. If the list is shuffled, the time complexity changes back to $O(n^2)$, and the optimized algorithm behaves like the non-optimized one.

3. Insertion Sort

3.1 Explanation of the algorithm

Insertion Sort is a simple sorting algorithm that sorts one element at a time. It can be compared with the technique of how cards are sorted at the time of playing a game. (Tutorials Point, n.d). The algorithm splits the list into two parts: a sorted part and an unsorted part. At the start, the sorted part only contains the first element. For each step, we take the next element from the unsorted part and place it in the correct position in the sorted part by shifting larger elements from one place to the right.

Algorithm Steps

1. Start with the second element (since a single-element list is already sorted).

2. Pick an element and compare it with the elements before it.
3. Insert the elements one position to the right to make space for the current element
4. Insert the element into its correct position.
5. Repeat the process for all elements in the list.

These steps are gathered directly from LO 3_SortingAlgorithms (Gupta, R (2026). LO 3_SortingAlgorithms, slide 6).

Pseudocode

The standard pseudocode for Insertion Sort is gathered from (Tutorials Point, n.d.)

```
Algorithm: Insertion-Sort(A)
for j = 2 to A.length
   key = A[j]
   i = j  1
   while i > 0 and A[i] > key
      A[i + 1] = A[i]
      i = i -1
   A[i + 1] = key
```

3.2 Code Implementation and Explanation

We implemented the Insertion Sort in the InsertionSort class in java. The sortUniqueAlcohol() method uses the CSVApplication class to load the wine dataset, filters out duplicate alcohol values using filereaderUnique(), and then passes the Array list to the insertionSort() method for sorting.

Kristiania

Source code

```
    public static int[] insertionSort(ArrayList<Wines> uniqueWines) {  2 usages
        int iterations = 0;
        int swaps = 0;

        for (int i = 1; i < uniqueWines.size(); i++) {
            Wines key = uniqueWines.get(i);
            int j = i - 1;

            while (j >= 0 && uniqueWines.get(j).alcohol > key.alcohol) {
                uniqueWines.set(j + 1, uniqueWines.get(j));
                j--;
                swaps++;
                iterations++;
            }
            uniqueWines.set(j + 1, key);
            iterations++;
        }

        System.out.println("Unique alcohol values: " + uniqueWines.size());
        for (Wines w : uniqueWines) {
            System.out.println("Alcohol: " + w.alcohol);
        }
        System.out.println("Iterations: " + iterations + " | Swaps: " + swaps);

        return new int[]{iterations, swaps};
    }

}
```

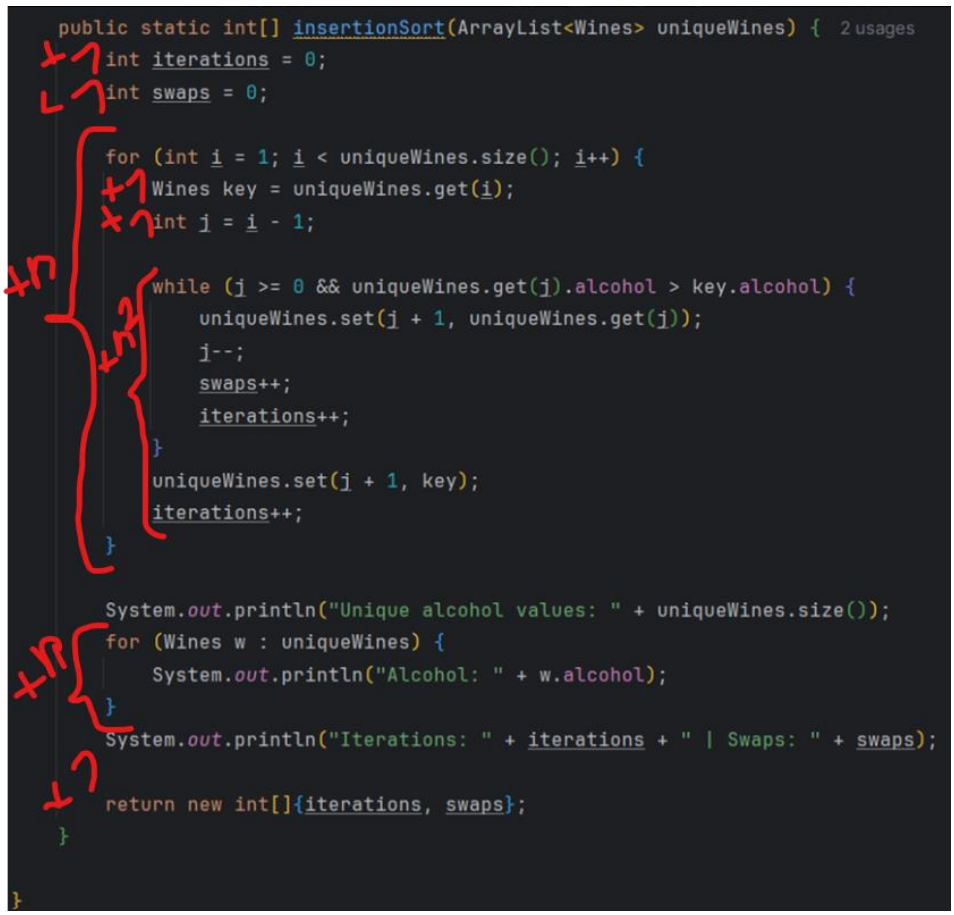
We will do step by step through the most important parts of the insertion Sort code:

- Since the loop starts at index 1, the element at (index 0) is already considered sorted. The loop then goes through the rest of the Array List uniqueWines one element at a time.
- We store the current element we want to place in the position to the right as the key using Wines key = uniqueWines.get(i).
- We then set j = i-1 to point to the last element of the sorted part. The loop then goes backwards through the sorted part to locate where the key fits.
- Our loop keeps going as long as both conditions are correct. Where j>=0 and that the element before the key has a larger alcohol content. When this is true, the element moves one step to the right to make room for the key.
- After the while loop stops, the key is placed in its correct position using uniqueWines.set(j+1, key).
- We have also added variables as iterations and swaps counters to keep track of how much our Insertion Sort does.

Kristiania

3.3 Calculation on time complexity

The screenshot below shows a complexity overview over our insertion sort; this demonstrates how parts of the code contribute to the total time complexity:



```
public static int[] insertionSort(ArrayList<Wines> uniqueWines) {  2 usages
    int iterations = 0;
    int swaps = 0;

    for (int i = 1; i < uniqueWines.size(); i++) {
        Wines key = uniqueWines.get(i);
        int j = i - 1;

        while (j >= 0 && uniqueWines.get(j).alcohol > key.alcohol) {
            uniqueWines.set(j + 1, uniqueWines.get(j));
            j--;
            swaps++;
            iterations++;
        }
        uniqueWines.set(j + 1, key);
        iterations++;
    }

    System.out.println("Unique alcohol values: " + uniqueWines.size());
    for (Wines w : uniqueWines) {
        System.out.println("Alcohol: " + w.alcohol);
    }
    System.out.println("Iterations: " + iterations + " | Swaps: " + swaps);

    return new int[]{iterations, swaps};
}
}
```

Here n is defined as the number of unique alcohol values stored in the Array list, which is 111 values from 6497 total rows. Our outer loop runs O(n) times. Therefore, the worst case would be that our inner loop runs up to n times for each step of the outer loop.
In our case, the values are fixed at 111 unique values from both csv files, so it will not grow. However, time complexity is a description of how the algorithm acts, which is not specific to our dataset. Our outer loop runs n-1, which runs through every element in our Array List, once. This gives us *O(n).* If our Array List of the data is already sorted, then our inner loop never runs since each element is already sorted in ascending order. This would give us the best case of *O(n)*, because only our outer loop runs. In the worst case, the number of shifts can be viewed with this formula:
$1 + 2 + ... + (n – 1) = n(n – 1)/2$
For each step of the outer loop, our inner loop can only run up to i times. This would therefore result in a time complexity of worst case defined as $O(n^2)$, since the order of elements in the Array List would be in a reversed order. The average case would still result in $O(n^2)$ time complexity, since we assume that the order of each element is random. This means that roughly each element needs

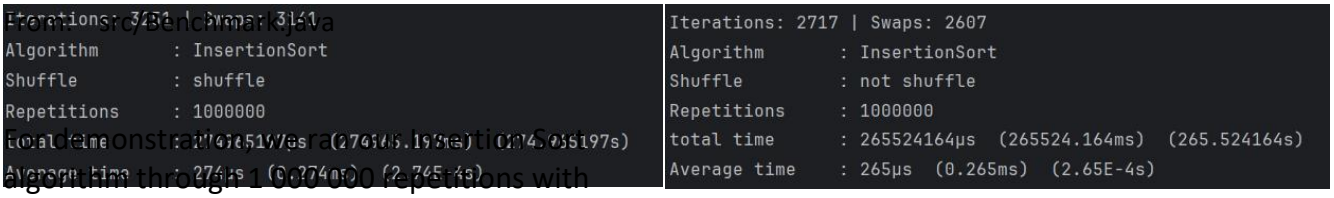
Kristiania

to be compared to half of the sorted part before being placed in the right spot GeeksforGeeks. (2024).

Best-case	Average-case	Worst-case
$O(n)$	$O(n^2)$	$O(n^2)$

3.4 Effect of Shuffling our Dataset

Task b asked if the time complexity changes if the dataset is randomly shuffled before sorting with Insertion Sort. The short answer to the question is that the Big-O complexity stays with the exception of the best case. Since the likelihood of best-case *O(n) is* more improbable if we were to shuffle the data. Shuffling the data could also change the actual run time of the algorithm, which we
will demonstrate below. Shuffling our Array List before sorting with Insertion Sort does not change the Big-O time complexity for both average- and worst case, since it is still $O(n^2)$. In the context of worst case, it will not change because Big-O describes the upper bound of how our algorithm scales.



From demonstrateInsertionSort()
algorithm through 1 000 000 repetitions with
and without shuffling. When we ran it without shuffling, we got 2717 iterations and 2607 swaps, with an average run time of 265 µs. With shuffling on, we got 3251 iterations and 3141 swaps, with an average run time of 274 µs. This confirms our hypothesis on the fact that the Big-O complexity stays the same, since shuffling only increases the number of iterations, swap, and run time.

3.5 conclusion

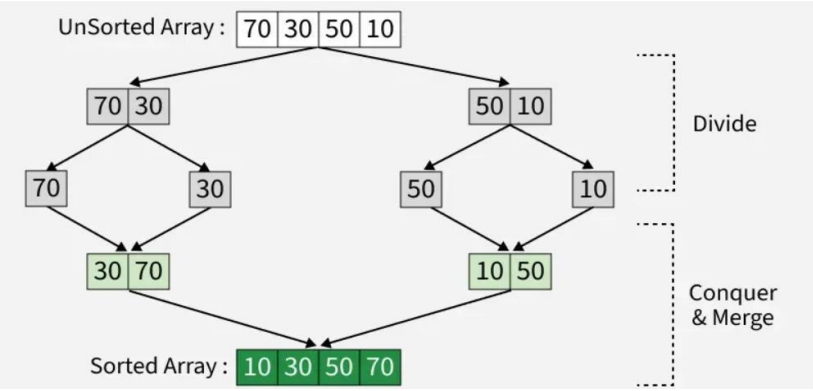
in summary, we have implemented a standard Insertion Sort following the source TutorialsPoint. (n.d.). We made it in order to sort the unique alcohol values from the combined white and red wine dataset in ascending order. Our algorithm works by building a sorted part of the joined Array List one value at a time by shifting higher value elements to the right until each new element can be sorted in the correct spot. When it comes to time complexity, best case of *O(n)* would be our dataset is already sorted. Average and worst case gives the same time complexity of $O(n^2)$, this occurs if the elements in the dataset were random or in reverse order. In addition, our data gathered from the benchmark class shows that randomly shuffling the wine dataset with 1 000 000 repetitions increases the average run time.

Kristiania

4. Merge Sort

4.1 Explanation of the algorithm

Merge sort is one of the most efficient sorting algorithms. It uses a divide-and-conquer-approach to sort the contents of a list or an array (Brilliant, 2026). How the divide and conquer strategy works is divided into three steps. We start off by dividing where the algorithm splits the array into two halves until each subarray has only one element (GeeksForGeeks, 2025). Then comes step two conquer, this is where the algorithm sorts each subarray individually with least to the left and highest to the right (GeeksForGeeks, 2025). For the step, the algorithm merges combining the now sorted subarrays into a single sorted list or array in an ascending order (GeeksForGeeks, 2025).
This can be easily visualized with this illustration:
credits to geeksforgeeks. 3. Oct, 2025. Link: https://www.geeksforgeeks.org/dsa/merge-sort/



This efficient sorting algorithm excels when it comes to sorting larger datasets since it faces the dataset head on making it smaller and later merging it back, for example, database systems (Sarkar, 2023). This algorithm comes especially handy when it comes to external sorting, where the data is too large to be stored into memory, but rather stored on disk later being divided into smaller chunks, sorted and merged (Youcademy, n.d.).

The problem with the Mergesort algorithm is that the algorithm has a recursive nature. What this means is that it is a recursive algorithm relying on repetition until the sorting is complete. This can lead to stack overflow errors when dealing with a large dataset (Sarkar, 2023). A con that can be a problem for a developer working with Mergesort can be its space complexity. Since Mergesort requires additional memory to store the merged subarrays after dividing the list (GeeksForGeeks, 2025). This can lead to unpleasant interactions when working in a very manual coding language like c in Linux where you must allocate the memory manually.

Kristiania

4.2 Code

For our code, we used GeeksForGeeks' version of Mergesort and adapted it to fit our dataset. This code is fairly simple, where the MergeSort method divides the list into smaller lists until they can no longer be split. After dividing the list, the merge method combines the smaller now sorted parts back. Later, the values are being printed leading to a sorted ascending list.
For the sake of the task, we have also implemented a counter for the amount of merge operations the algorithm has achieved.

```java
import java.util.ArrayList;


public class MergeSort {  3 usages
    ////////////////////////////////////////////////////
    /// author: GeeksForGeeks                        ///
    /// gathered: 5.04.2026                          ///
    /// url: https://www.geeksforgeeks.org/dsa       ///
    /// /merge-sort/                                 ///
    /// title: Merge Sort                            ///
    /// last updated: 3 oct, 2025                    ///
    ////////////////////////////////////////////////////

    //L is Left
    //M is Mid
    //R is Right
    //shortend for efficiency reasons

    static int counter = 0;  4 usages

    public static void mergeSort(ArrayList<Wines> wineList, int L, int R) {  4 usages
        if (L < R) {
            int M = L + (R - L) / 2;

            mergeSort(wineList, L, M);
            mergeSort(wineList,  L: M + 1, R);

            merge(wineList, L, M, R);
        }
```

From: src/MergeSort.java

Kristiania

```java
    public static void merge(ArrayList<Wines> list, int L, int M, int R) {  1 usage
        counter++;

        int n1 = M - L + 1;
        int n2 = R - M;

        ArrayList<Wines> tempL = new ArrayList<>();
        ArrayList<Wines> tempR = new ArrayList<>();

        for (int i = 0; i < n1; i++) {
            tempL.add(list.get(L + i));
        }
        for (int j = 0; j < n2; j++) {
            tempR.add(list.get(M + 1 + j));
        }
        int i = 0, j = 0;

        int k = L;
            while (i < n1 && j < n2) {
                if (tempL.get(i).alcohol <= tempR.get(j).alcohol) {
                    list.set(k, tempL.get(i));
                    i++;
                } else {
                    list.set(k, tempR.get(j));
                    j++;
                }
                k++;
            }
            while (i < n1) {
                list.set(k, tempL.get(i));
                i++;
                k++;
            }
            while (j < n2) {
                list.set(k, tempR.get(j));
                j++;
                k++;
            }
        }
    public static void sortUniqueAlcohol() {  1 usage
        counter = 0;
        CVSApplication csvApp = new CVSApplication();
        ArrayList<Wines> uniqueWines = csvApp.filereaderUnique();
        mergeSort(uniqueWines,  L: 0,  R: uniqueWines.size() - 1);
        System.out.println("unique alcohol values:" + uniqueWines.size());

        for (Wines w : uniqueWines) {
            System.out.println("Alcohol: " + w.alcohol);
        }

        System.out.println("Number of merge operations: " + counter);
        counter=0;
    }
}
```

From: src/MergeSort.java

Kristiania

4.3 Counting of merge operations

Count the number of merge operations needed to sort the dataset?

To count the number of operations we have implemented a counter (counter = 0;) into our sortUniqueAlcohol void. This counter starts at zero once the merge method has been called upon, then the counter increases (counter++ (in the merge method) later showing us the amount of merge operations performed by the algorithm. After the values of the sorted list and the amount of merge operations, the algorithm has performed the counter resets (counter = 0; after the System.out.println;). The value presented is 110 merge operations.

```
Alcohol: 14.9
Number of merge operations: 110
Time: 137.0000 ms
Time (ms); 137105400ms
```

note that the time remains as is because the algorithm has not been run a couple of time as warmup

Does this number change if you randomly shuffle the list before sorting? Why or why not?

The amount of merge operations stays the same regardless of whether the list being shuffled or not. This is caused by the fact that the list is always divided in the same way based on its size. This is also solidified by the fact that the value presented by the sorted shuffled mergesort is also 110 merge operations.

```
Alcohol: 14.9
Merge operations: 110
```

4.4 conclusion

To conclude, mergesort is a vast and effective algorithm that can be used in large and complex operations. This includes but is not limited to large data sorting and external sorting. This algorithm does have certain flaws, but it still remains a fairly reliable sorting algorithm. When it comes to the number of merging operations, it remains the same no matter if it is shuffled or not. What affects the number of merge operations is in fact the scale and size of the data list.

Kristiania

When it comes to the sorting of wine list, this algorithm proves extremely efficient and reliable. The time remains fairly the same after a couple of runs (after warmup). This is solidified by the output after being run 5 times in a row.

Run 1:

```
Number of merge operations: 110
Time: 27.0000 ms
Time (ms); 27657000ms
```

Run 2:

```
Number of merge operations: 110
Time: 26.0000 ms
Time (ms); 26224500ms
```

Run 3:

```
Number of merge operations: 110
Time: 25.0000 ms
Time (ms); 25712800ms
```

Run 4:

```
Number of merge operations: 110
Time: 24.0000 ms
Time (ms); 24565200ms
```

Run 5:

```
Number of merge operations: 110
Time: 25.0000 ms
Time (ms); 25515000ms
```

As the results show, the mergesort algorithm remains reliable to the wine list dataset.

Kristiania

5. Quick Sort

5.1 Explanation of the algorithm

Quicksort is a very efficient algorithm that uses the same strategy of divide and conquer as mergesort. What sets apart these two sorting algorithms is the fact that quicksort is designed for speed at the same time using divide and conquer (Wikipedia, 2026). Quicksort works on the principle of choosing a pivot and with that pivot to divide and conquering the list.
The speed of the sorting now depends on what element has been chosen as the pivot (GeeksForGeeks, 2025. And Goodrich. (Page 544, 545)).
The algorithm follows four steps to sort a list presented to it.
Step 1. Choosing a pivot:
The algorithm selects an element from the array as the pivot. This pivot can be different elements in the list / array. This includes first last, random element or median (GeeksForGeeks. 2025). The pivot can affect how fast the algorithm sorts the given list or array.

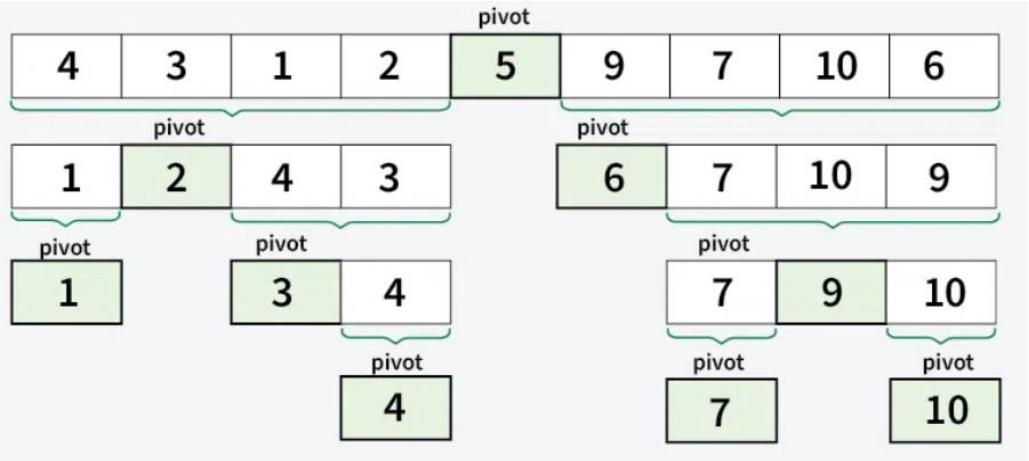
Step 2. Partitioning the array/list:
In the next step the algorithm re arranges the list/array around the pivot. After splitting the list/array all the lesser elements end up on the left of the pivot, and all the greater elements end up on the right of the pivot (GeeksForGeeks. 2025).

Step 3. Recursively Calling:
Later the algorithm applies the same process to the split sub array (GeeksForGeeks. 2025).

Step 4. Base Ceasing:
For the last step when the recursion stops when there is only one element in the sub array. When there is only a single element left, the list is sorted (GeeksForGeeks. 2025)

This can be better visualized by this illustration:

credits to geeksforgeeks. 8. Dec, 2025. Link: https://www.geeksforgeeks.org/dsa/quick-sort-algorithm/



Kristiania

When it comes to advantages with quicksort, efficiency is the first attribute that pops up. Since quicksort has an average time complexity of O (n log n), it makes quicksort one of the fastest algorithms out there (Youcademy n.d.). Another great aspect of quicksort is the fact that even if the list size (n) grows, the time it takes to sort increases in a drastically lower phase than other algorithms (Youcademy n.d.). In contrast to merging, sort quicksort is much favorable to developers due to quicksort no need for extra memory for merging (Youcademy n.d.). This comes very hand if you code in languages like C where memory allocation is manual and bugs may come up due to memory loss

There are not a lot of disadvantages to quicksort, but there are problems due to its complexity and the fact that the algorithm is advanced may lead to some problems. Due to its advanced code, it can be harder to implement quicksort correctly without having error or bugs. Compared to simpler algorithms, quicksort may cause problems for developers (Goswami. 2025). Another problem with quicksort is the choice of a bad pivot can lead to poor performance and unbalanced partitioning (Goswami. 2025)

Kristiania

5.2 Code

Our Quicksort is just like our Bubblesort and Mergesort, a modified version of the GeeksForGeeks' Quicksort that we adapted to our wine list. Just like the previous algorithms, it follows a similar structure, but where this code differs is the usage of divide and conquer strategy that is implemented into the code with the use of several methods. Our quicksort code consists of the main Sort void which calls upon the quicksort void, and the quicksort void uses helper voids partition void. The partition void benefits from a switch that chooses which pivot mode the quicksort will use. The partition void also benefits from the medianofthree int and swap void.

The quicksort algorithm is quite complex. We have tried to implement it with our own names to make it a bit simpler for us to understand this process. This way we can learn the algorithm better.

```java
import java.util.ArrayList;
import java.util.Random;

public class QuickSort {  4 usages
    static int counter = 0;  3 usages


    ////////////////////////////////////////////////////
    /// author: GeeksForGeeks                        ///
    /// gathered: 5.04.2026                          ///
    /// url: https://www.geeksforgeeks.org/dsa/      ///
    /// quick-sort-algorithm/                        ///
    /// title: Merge Sort                            ///
    /// last updated: 3 oct, 2025                    ///
    ////////////////////////////////////////////////////

    public static void quickSort(ArrayList<Wines> wineList, int low, int high, int m) {  3 usages
        if (low < high) {
            int pi = partition(wineList, low, high, m);

            quickSort(wineList, low,  high: pi - 1, m);
            quickSort(wineList,  low: pi + 1, high, m);
        }
    }
    public static int partition(ArrayList<Wines> wineList, int low, int high, int m) {  1 usage
        //pivoting m = modes.
        switch (m) {
            //chosing the first element
            case 1:
                swap(wineList, low, high);
                break;
                //chosing last element.
                case 2:
                    break;
                    //chosing random element.
                    case 3:
                        Random r = new Random();
                        int random = low + r.nextInt( bound: high - low + 1);
                        swap(wineList, random, low);
                        break;
                        //chosing median of three
                        case 4:
                            int mid = (low + high) / 2;
                            int median = medianOfThree(wineList, low, mid, high);
                            swap(wineList, median, high);
                            break;
        }
```

Kristiania

```
        double pivot = wineList.get(high).alcohol;
        int i = low - 1;
        for (int j = low; j < high; j++) {
            counter++; // to count the number of comparison
            if (wineList.get(j).alcohol < pivot) {
                i++;
                swap(wineList, i, j);
            }
        }
        swap(wineList, i: i + 1, high);
        return i + 1;
    }

    public static int medianOfThree(ArrayList<Wines> wineList, int i, int j, int n) { 1 usage
        double a = wineList.get(j).alcohol;
        double b = wineList.get(i).alcohol;
        double c = wineList.get(j).alcohol;

        if ((a > b && a < c) || (a < b && a > c)) return i;
        if ((b > a && b < c) || (b < a && b > c)) return j;
        return n;
    }
    public static void swap(ArrayList<Wines> wineList, int i, int j) { 5 usages
        Wines w = wineList.get(i);
        wineList.set(i, wineList.get(j));
        wineList.set(j, w);
    }
    public static void sort(int m) { 4 usages
        CVSApplication CVSapp = new CVSApplication();
        ArrayList<Wines> wineList = CVSapp.filereaderUnique();

        counter = 0; // just a simple reset before running

        quickSort(wineList, low: 0, high: wineList.size() - 1, m);

        for (Wines w : wineList) {
            System.out.println("Alcohol: " + w.alcohol);
        }
        System.out.println("Comparisons: " + counter);

    }
    // to call upon element first
    public static void QSFirst() { 1 usage
        sort( m: 1);
    }
    // to call upon element last
    public static void QSLast() { 1 usage
        sort( m: 2);
    }
    // to call upon random element
    public static void QSRandom() { 1 usage
        sort( m: 3);
    }

    // to call upon median of three
    public static void QSMedianThree() { 1 usage
        sort( m: 4);
    }
}
```

From: src/QuickSort.java

Kristiania

5.3 Quick sort pivoting and its strategies

Pivoting is a large part of quicksort; choosing the right pivot is essential to the performance of the Quicksort Algorithm (Ayush. 2019). I shall demonstrate the difference between four different pivoting factors. There are in total four pivoting modes included in our examination task. These quicksort pivoting modes include the first element in the list, the last element in the list, a random element in the list, and a median of three elements in the list.
We have implemented all four of these modes into our application, and it can be chosen which pivoting mode you wish to proceed with. There is also a comparison counter implemented to count the comparisons based on the pivot strategy.

a. Does the number of comparisons change depending on the pivot strategy?
During our research we have concluded that the number of comparisons does in fact change depending on the pivot's strategy. The reason behind that is the fact just like mentioned earlier. A good pivot that has a balanced spilt will in fact have less recursive calls that will lead to fewer comparisons just like how a bad pivot will lead to a bad spilt which will lead to more comparisons (GeeksForGeeks. 2025).

This is further solidified by the outputs of our application.
The comparison counter during the pivot mode first element in the list is 740 comparisons per sort.

```
Comparisons: 740
Time: 39.0000 ms
Time (ms); 39802800ms
```

The comparison counter for the pivot mode last element in the list has a counter of 685 comparisons per sort.

```
Comparisons: 685
Time: 111.0000 ms
Time (ms); 111051000ms
```

The comparison counter for the pivot mode random element in the list has a counter of 665 (for the first run).

```
Comparisons: 665
Time: 35.0000 ms
Time (ms); 35797500ms
```

Note that this number may vary drastically due to the fact that the pivot is random. This means that the split could either be very balanced or very unbalanced based on which element is chosen. With a few more tests we have concluded that the best comparisons counter landed on was 657 and the worst was 729. This shows very clearly the difference in comparisons once there is a good pivot chosen randomly and a bad pivot chosen randomly.
For the last pivot mode, which is the median of three in the list, it has a comparison counter of 665 per sort.

```
Comparisons: 665
Time: 182.0000 ms
Time (ms); 182458600ms
```

b. Which pivots selection strategy works best for this dataset, and why?

Earlier we concluded the fact that the first element in the list as the pivot has the largest comparison

number out of all the four pivot modes with a count of 740 comparisons. This prompted us to put the pivot mode first element in the list as last place in this scenario. Then for the second worst pivot mode we would argue would be the random element in the list. The reason behind this decision is the fact that this pivoting mode is way too unpredictable and can lead to either a very good and balanced split or a very bad and unbalanced spilt. Then, for the second-best pivoting mode, there is the last element in the list with the comparison count of 685. Now for the best pivoting mode there is the median of three with the comparison count of 665, in general the median of three is the most balanced pivot out of all the four pivoting modes due to the fact that it often will make a balanced split in a large amount of lists not just this list.

5.4 Conclusion

To conclude, Quicksort is in a way faster version than mergesort but mergesort guarantees more stable performance. It uses divide and conquers, but it is faster and uses less data. Although it is more unstable than mergesort (due to the need to choose the right pivot) it outclasses the other algorithms in this application due to its speed and efficiency. We argue that quicksort is the best algorithm for this dataset due to the fact that the wine list is a small dataset that can be sorted incredibly fast. Even if the list was larger, quicksort would still be the best choice in our opinion.

Kristiania

6. Benchmark

For our benchmark application, we have tried to be creative while implementing the essential codes that let us run our algorithms multiple times. This application ensures to show performance of every algorithm in the menu application. How this code works is it first loads the data and repeatedly sorts it until it reaches the chosen repetitions, and for each run it times how long the sorting takes. We have also added shuffle as a parameter to the run method, which allows the user to test the performance with and without shuffling. This application can also calculate the total and average time after sorting to compare which algorithm is faster.

Underneath is the code of our benchmark application:

```java
import java.util.ArrayList;
import java.util.Collections;

public class WineSortingBenchmark {  1 usage

    public static void run(){  1 usage
        CVSApplication application = new CVSApplication();
        ArrayList<Wines> wines = application.filereaderUnique();

        System.out.println("----------Benchamrk Starting ---------");
        System.out.println("Dataset size: " + wines.size() + "unique alcohol values");

        // Both optimised and un-optimised benchmarks for BubbleSort with and without shuffle

    //    benchBubbleSortUnOpt(wines, false, 100000);
    //    benchBubbleSortUnOpt(wines, false, 1000000);
        benchBubbleSortOpt(wines,  shuffle: false,  repetitions: 1000000);
    //    benchBubbleSortOpt(wines, true, 100);

        // Benchmark for InsertionSort with and without shuffle
    //    benchInsertionSort(wines, false, 1000000);
    //    benchInsertionSort(wines, true, 100000);

        //Benchmark for MergeSort with and without shuffle
    //     benchMergeSort(wines, false, 100000);
    //    benchMergeSort(wines, true, 100000);
```

Some sections are commented out so that specific methods can be run. This is due to the fact that some segments are meant to be run separately from each other.

```java
    //Benchmark for QucikSort with and without shuffle, and with number of comparisons per pivot strategy
    //    benchQuickSort(wines, false, 10000, 1, "QuickSort (first element pivot )");
    //    benchQuickSort(wines, false, 10000, 2, "QuickSort (last element pivot )");
    //    benchQuickSort(wines, false, 10000, 3, "QuickSort (random  element pivot )");
    //    benchQuickSort(wines, false, 10000, 4, "QuickSort (median element pivot )");
    }

    private static void benchBubbleSortUnOpt(ArrayList<Wines> wines, boolean shuffle, int repetitions) {  no usages
        System.out.println("----------Benchmark BubbleSort (non-optimised) Starting ---------");
        Timer timer = new Timer();
        long totalTime = 0;

        for (int i = 0; i < repetitions; i++) {
            ArrayList<Wines> copy = new ArrayList<>(wines);
            if (shuffle) Collections.shuffle(copy);

            timer.start();
            BubbleSort.bubbleSortUnOpt(copy);
            timer.end();

            totalTime += timer.getElapsedTime() / 1000;
        }
```

This is the benchmark for the nonoptimized bubblesort. Later, this benchmark gets timed.

Kristiania

```
        timeResult( name: "BubbleSort (non-optimised)", shuffle, repetitions, totalTime);
    }
    private static void benchBubbleSortOpt(ArrayList<Wines> wines, boolean shuffle, int repetitions) {  1 usage
        System.out.println("----------Benchmark BubbleSort (optimised) Starting ---------");
        Timer timer = new Timer();
        long totalTime = 0;

        for (int i = 0; i < repetitions; i++) {
            ArrayList<Wines> copy = new ArrayList<>(wines);
            if (shuffle) Collections.shuffle(copy);

            timer.start();
            BubbleSort.bubbleSortOpt(copy);
            timer.end();

            totalTime += timer.getElapsedTime() / 1000;
        }

        timeResult( name: "BubbleSort (optimised)", shuffle, repetitions, totalTime);
    }
```

This is the benchmark method for the optimized bubblesort. Later, the benchmark gets timed.

```
    private static void benchInsertionSort(ArrayList<Wines> wines, boolean shuffle, int repetitions) {  no usages
        System.out.println("----------Benchmark IntersionSort (non-optimised) Starting ---------");
        Timer timer = new Timer();
        long totalTime = 0;

        for (int i = 0; i < repetitions; i++) {
            ArrayList<Wines> copy = new ArrayList<>(wines);
            if (shuffle) Collections.shuffle(copy);

            timer.start();
            InsertionSort.insertionSort(copy);
            timer.end();

            totalTime += timer.getElapsedTime() / 1000;
        }

        timeResult( name: "InsertionSort", shuffle, repetitions, totalTime);
    }
```

This is the benchmark method for the Insertionsort. Later, the benchmark gets timed.

```
    private static void benchMergeSort(ArrayList<Wines> wines, boolean shuffle, int repetitions) {  no usages
        System.out.println("----------Benchmark MergeSort Starting ---------");
        Timer timer = new Timer();
        long totalMergeOpn = 0;
        long totalTime = 0;


        for (int i = 0; i < repetitions; i++) {
            ArrayList<Wines> copy = new ArrayList<>(wines);
            if (shuffle) Collections.shuffle(copy);
            MergeSort.counter = 0;

            timer.start();
            MergeSort.mergeSort(copy,  L: 0,  R: copy.size() - 1);
            timer.end();

            totalTime += timer.getElapsedTime() / 1000;
            totalMergeOpn += MergeSort.counter;
        }
        timeResult( name: "MergeSort", shuffle, repetitions, totalTime);
        System.out.println("Average MergeSort operations over " + repetitions + " test: " + (totalMergeOpn / repetitions));
    }
```

This is the method for the mergesort. Later, the benchmark gets timed.

Kristiania

```
114     private static void benchQuickSort(ArrayList<Wines> wines, boolean shuffle, int repetitions, int pivotMode, String name) {  no usages
115         System.out.println("----------Benchmark QuickSort Starting ---------");
116         Timer timer = new Timer();
117         long totalComparisions= 0;
118         long totalTime = 0;
119
120         for (int i = 0; i < repetitions; i++) {
121             ArrayList<Wines> copy = new ArrayList<>(wines);
122             if (shuffle) Collections.shuffle(copy);
123             QuickSort.counter = 0;
124
125             timer.start();
126             QuickSort.quickSort(copy,  low: 0,  high: copy.size() - 1, pivotMode);
127             timer.end();
128
129             totalTime += timer.getElapsedTime() / 1000;
130             totalComparisions += QuickSort.counter;
131         }
132         timeResult(name, shuffle, repetitions, totalTime);
133         System.out.println("Average comparions over " + repetitions + " test of " + name + ": " + (totalComparisions / repetitions));
134     }
```

This is the method for quicksort. Later the benchmark gets timed

```
136     private static void timeResult(String name, boolean shuffle, int repetitions, long totalTime){  5 usages
137         String shuffleLabel;
138         if (shuffle) {
139             shuffleLabel = "shuffle";
140         } else {
141             shuffleLabel = "not shuffle";
142         }
143         long avgTime = totalTime / repetitions;
144
145         System.out.println("Algorithm       : " + name);
146         System.out.println("Shuffle         : " + shuffleLabel);
147         System.out.println("Repetitions     : " + repetitions);
148         System.out.println("total time      : " + totalTime + "µs  (" + (totalTime /
149                 1000.0) + "ms)  (" + (totalTime / 1000000.0) + "s)" );
150         System.out.println("Average time    : " + avgTime + "µs  (" + (avgTime /
151                 1000.0) + "ms)  (" + (avgTime / 1000000.0) + "s)");
152
153
154
155
156     }
157 }
```

In the end after a selected method has been run and the algorithm of choice has been benchmarked the value gets presented and outputted.

Kristiania

7. References and misc.

7.1 References

Intro

Bharat. (2024, April 3). *What is ArrayList in Java and how to create and all predefined methods and different ways to print ArrayList element?* https://medium.com/@Bharat2044/what-is-arraylist-in-java-and-how-to-create-and-all-predefined-methods-and-different-ways-to-print-fa3cc9f86f8b

GeeksforGeeks. (2025, August 5). *Top programming languages for competitive programming*. https://www.geeksforgeeks.org/blogs/top-programming-languages-for-competitive-programming/

GeeksforGeeks. (2025, July 23). *Java program for bubble sort*. https://www.geeksforgeeks.org/dsa/java-program-for-bubble-sort/

GeeksforGeeks. (2026, January 20). *Stack data structure*. https://www.geeksforgeeks.org/dsa/stack-data-structure/

GeeksforGeeks. (2026, January 20). *Queue data structure*. https://www.geeksforgeeks.org/dsa/queue-data-structure/

GeeksforGeeks. (2025, July 23). *Array vs ArrayList in Java*. https://www.geeksforgeeks.org/java/array-vs-arraylist-in-java/

GeeksforGeeks. (2026, March 3). *Array data structure*. https://www.geeksforgeeks.org/dsa/array-data-structure-guide/

Gupta, R (2026). LO 1_SortingAlgorithms, slide 9 and 11. Kristiania collage of applied science

Gupta, R (2026). LO 1_SortingAlgorithms, slide 3. Kristiania collage of applied science

Ryan Mcgregor (2025, August 5). *Difference between array and ArrayList in Java – the ultimate guide*. https://ruby-doc.org/blog/difference-between-array-and-arraylist-in-java/

Shravya (2024, July 6) Resizing arrays in java.
https://medium.com/@shras_a/resizing-arrays-eb69888bc821

W3Schools (n.d) *Java ArrayList.*
https://www.w3schools.com/java/java_arraylist.asp

Bubblesort

CodeGym. (2025, January 9). *Bubble sort java*. https://codegym.cc/groups/posts/bubble-sort

GeeksforGeeks. (2025, December 8). *Bubble sort algorithm*. https://www.geeksforgeeks.org/dsa/bubble-sort-algorithm/

AlmaBetter. (2026, February 25). *When to use bubble sort (and when not to)*. https://www.almabetter.com/bytes/tutorials/tutorial-for-bubble-sort-in-java/when-to-use-bubble-sort

W3Schools. (n.d.). *DSA Time complexity theory*. https://www.w3schools.com/dsa/dsa_timecomplexity_theory.php

GeeksforGeeks. (2026, March 14). *Space Complexity.* https://www.geeksforgeeks.org/dsa/g-fact-86/

GeeksforGeeks. (2026, Februar 24). *Insertion sort algorithm*. https://www.geeksforgeeks.org/insertion-sort-algorithm/

TutorialsPoint. (n.d.). *Insertion sort algorithm*. https://www.tutorialspoint.com/data_structures_algorithms/insertion_sort_algorithm.htm

Gupta, R (2026). LO 3_SortingAlgorithms, slide 4 and 5. Kristiania collage of applied science.

Mergesort

Brilliant.org. (2026, April 19). *Merge sort*.
https://brilliant.org/wiki/merge/
Youcademy. (n.d.). *Merge sort pros and cons*.
https://youcademy.org/merge-sort-pros-cons/
Sarkar, S. (2023, April 3). *Mastering merge sort: Pros, cons, and real-life applications in Python*. Medium. https://surajsarkar0.medium.com/mastering-merge-sort-pros-cons-and-real-life-applications-in-python-5ee4ae875eaf
GeeksforGeeks. (2025, October 3). *Merge sort*.
https://www.geeksforgeeks.org/dsa/merge-sort/

Quicksort

GeeksforGeeks. (2025, December 8). *Quick sort*.
https://www.geeksforgeeks.org/dsa/quick-sort-algorithm/
Youcademy. (n.d.). Advantages and disadvantages of quick sort algorithm.
https://youcademy.org/quick-sort-advantages-disadvantages/
Goswami, N. (2025, August 6). *Quicksort algorithm: Working, time complexity & advantages*.
https://www.iquanta.in/blog/quicksort-algorithm-working-time-complexity-advantages/
Arora, A. (2019, December 18). *Sorting algorithms: Properties/ pros/ cons/ comparisons*.
https://ayush-arora.medium.com/sorting-algorithms-properties-pros-cons-comparisons-1b43599d587b
Wikipedia contributors. (2026, April 15). *Quicksort*. Wikipedia.
https://en.wikipedia.org/wiki/Quicksort
Goodrich, M. T. (2015). *Data structures and algorithms in Java* (6th ed.). Wiley.